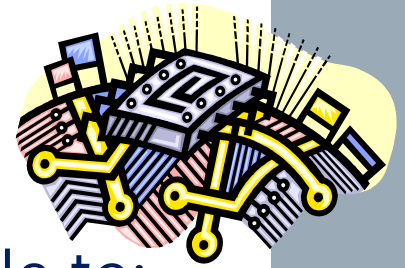


# Chapter 7

## Processor Structure and Function



# Objectives



After studying this chapter, you should be able to:

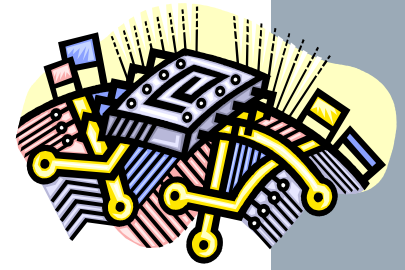
- › Distinguish between user-visible and control/status registers, and discuss the purposes of registers in each category.
- › Summarize the instruction cycle.
- › Discuss the principle behind instruction pipelining and how it works in practice.
- › Compare and contrast the various forms of pipeline hazards (rủi ro).



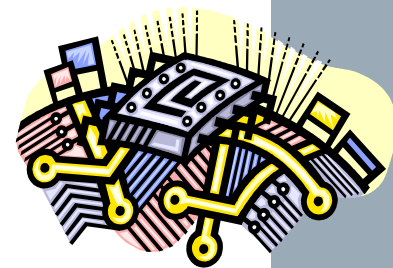
# Contents

$\pi$

- › 14.1 Processor Organization
- › 14.2 Register Organization
- › 14.3 Instruction Cycle
- › 14.4 Instruction Pipelining



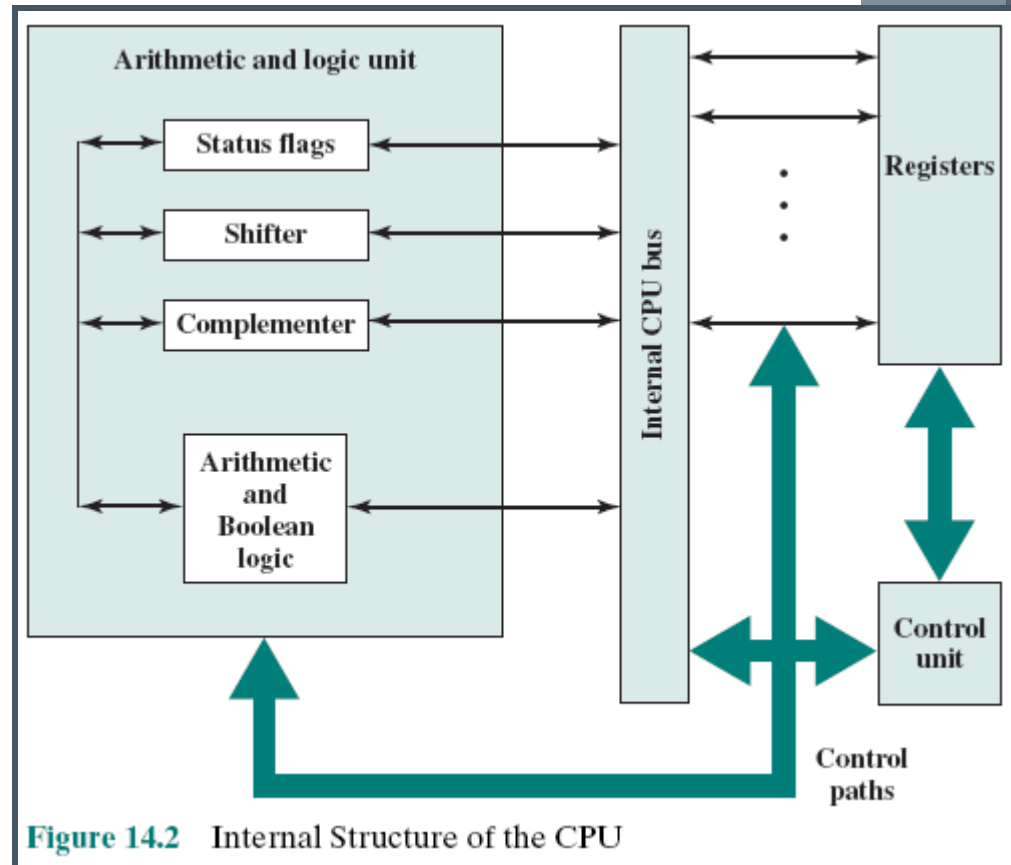
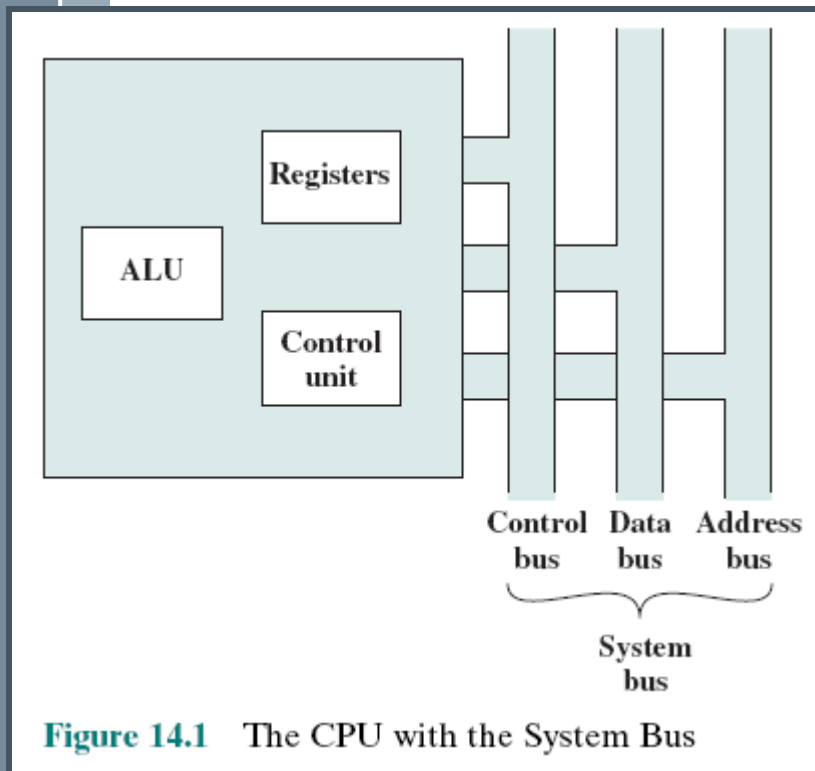
## 14.1- Processor Organization



### Processor Requirements:

- › Fetch instruction (from memory (register, cache, main memory))
- › Interpret instruction (what action is required)
- › Fetch data (data from memory or an I/O module)
- › Process data (performing some operations on data)
- › Write data (writing result to memory or an I/O module)
- ➔ In order to do these things the processor needs to store some data temporarily and therefore needs a small internal memory

# CPU With the System Bus and CPU Internal Structure



## 14.2- Register Organization

■  $\pi$

Within the processor there is a set of registers that function as a level of memory above main memory and cache in the hierarchy

The registers in the processor perform two roles:

### USER-VISIBLE REGISTERS

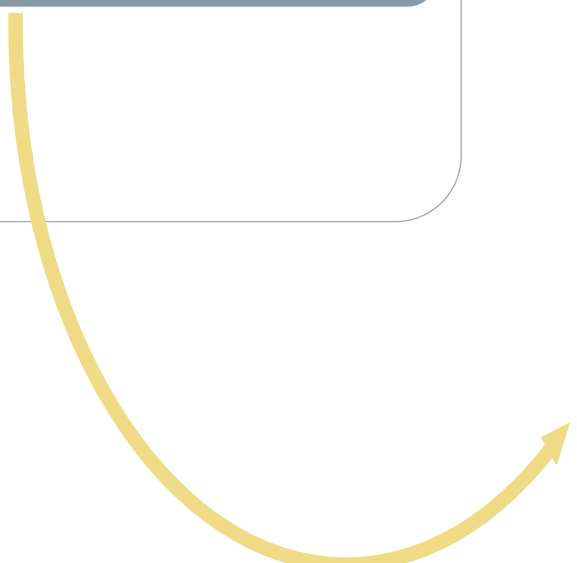
- › Enable the machine or assembly language programmer to minimize main memory references by optimizing use of registers

### CONTROL AND STATUS REGISTERS

- › Used by the control unit to control the operation of the processor and by privileged operating system programs to control the execution of programs

# User-Visible Registers

Referenced by means of the machine language that the processor executes



## Categories:

- **General purpose**
  - Can be assigned to a variety of functions by the programmer
- **Data**
  - May be used only to hold data and cannot be employed in the calculation of an operand address
- **Address**
  - May be somewhat general purpose or may be devoted to a particular addressing mode
  - Examples: segment pointers, index registers, stack pointer
- **Condition codes**
  - Also referred to as *flags*
  - Bits set by the processor hardware as the result of operations

# Table 14.1: Condition Codes

**Table 14.1** Condition Codes

| Advantages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Disadvantages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. Because condition codes are set by normal arithmetic and data movement instructions, they should reduce the number of COMPARE and TEST instructions needed.</li><li>2. Conditional instructions, such as BRANCH are simplified relative to composite instructions, such as TEST AND BRANCH.</li><li>3. Condition codes facilitate multiway branches. For example, a TEST instruction can be followed by two branches, one on less than or equal to zero and one on greater than zero.</li><li>4. Condition codes can be saved on the stack during subroutine calls along with other register information.</li></ol> | <ol style="list-style-type: none"><li>1. Condition codes add complexity, both to the hardware and software. Condition code bits are often modified in different ways by different instructions, making life more difficult for both the microprogrammer and compiler writer.</li><li>2. Condition codes are irregular; they are typically not part of the main data path, so they require extra hardware connections.</li><li>3. Often condition code machines must add special non-condition-code instructions for special situations anyway, such as bit checking, loop control, and atomic semaphore operations.</li><li>4. In a pipelined implementation, condition codes require special synchronization to avoid conflicts.</li></ol> |





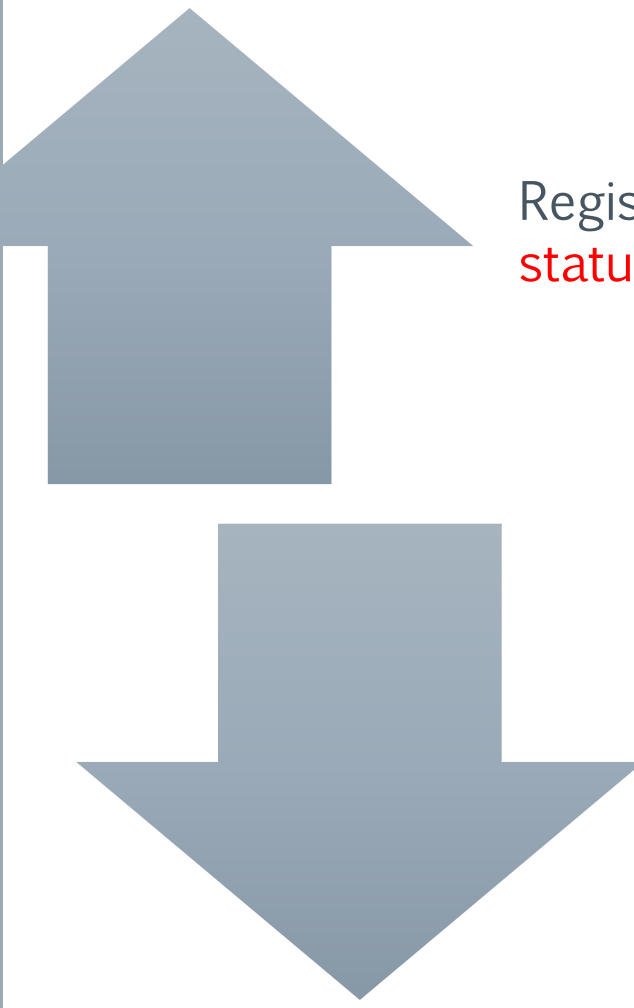
# Control and Status Registers

Four registers are essential to instruction execution:

- › **Program counter (PC)**
  - Contains the address of an instruction to be fetched
- › **Instruction register (IR)**
  - Contains the instruction **most recently fetched**
- › **Memory address register (MAR)**
  - Contains the address of a location in memory
- › **Memory buffer register (MBR)**
  - Contains a word of data to be written to memory or the word most recently read

# Program Status Word (PSW)

$\pi$

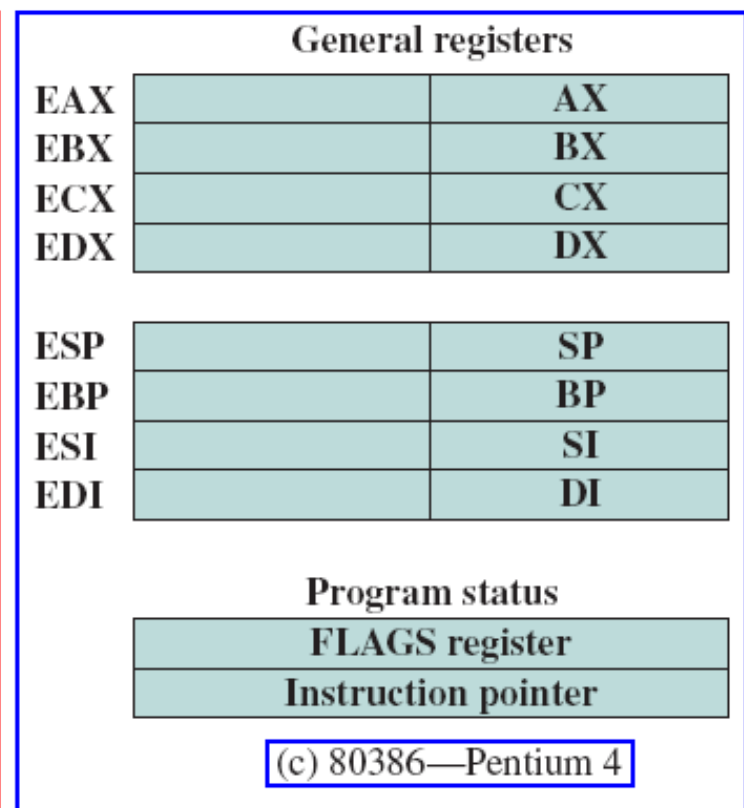
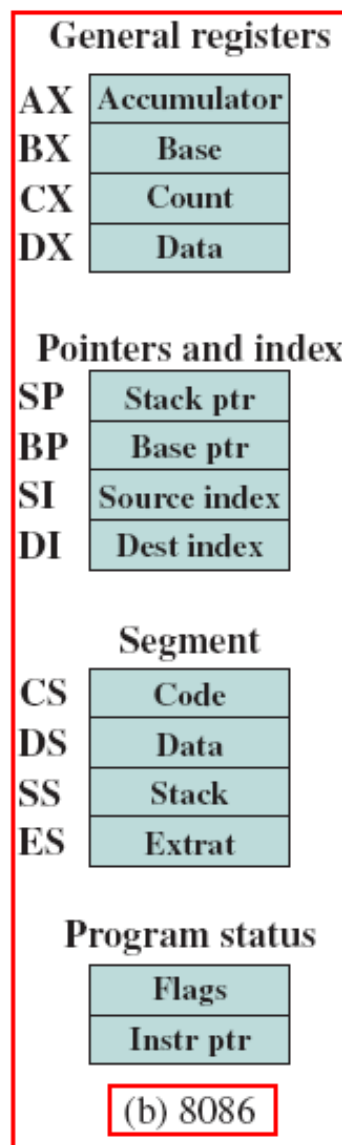


Register or set of registers that contain  
**status information**

Common fields or flags include

- Sign
- Zero
- Carry
- Equal
- Overflow
- Interrupt Enable/Disable
- Supervisor

Status information  
are used to give a  
decision for  
branching



# Example Microprocessor Register Organizations

**Figure 14.3** Example Microprocessor Register Organizations

# 14.3- Instruction Cycle

Includes the following stages:

Fetch

Read the **next instruction** from memory into the processor

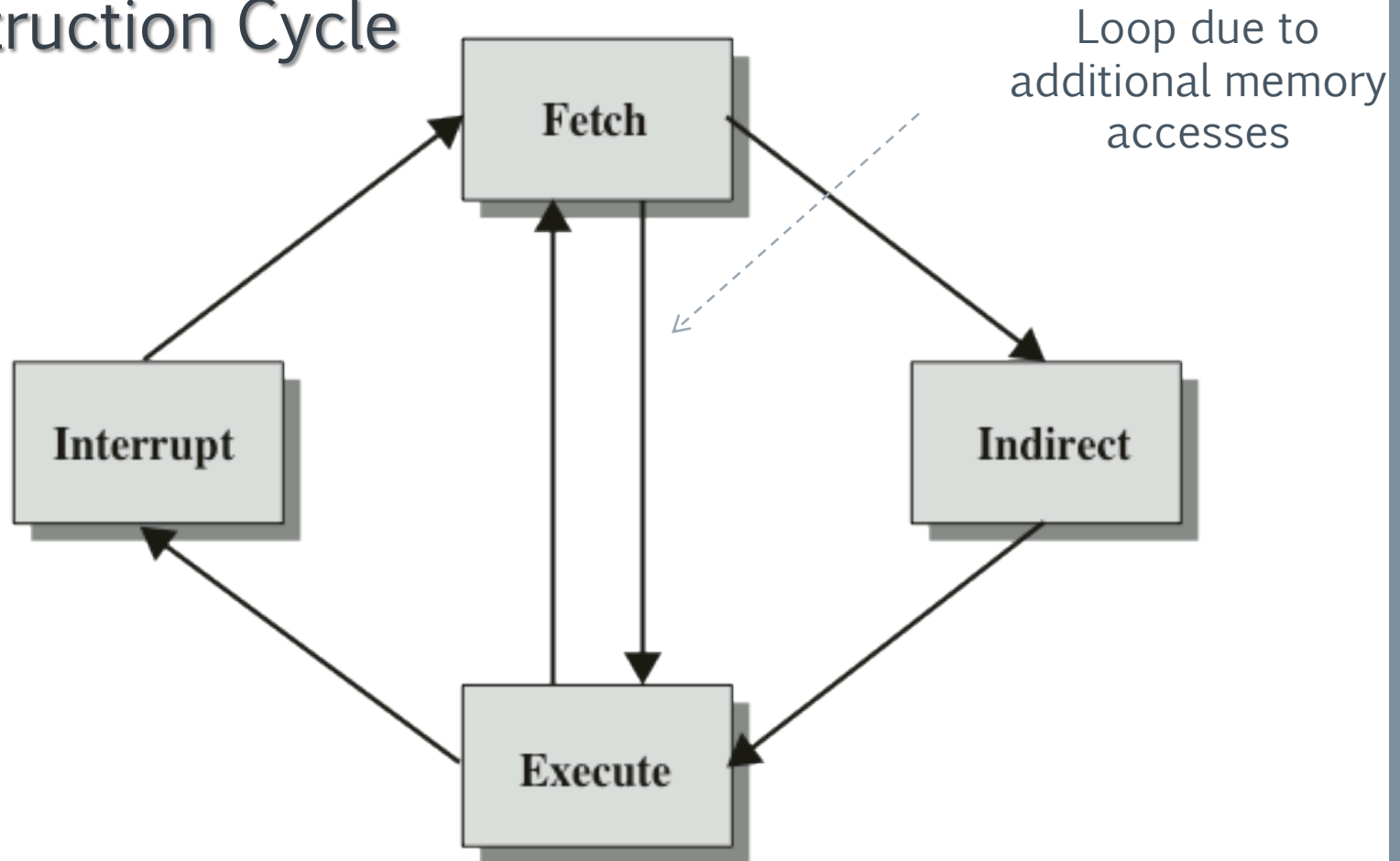
Execute

Interpret the **opcode** and **perform** the indicated operation

Interrupt

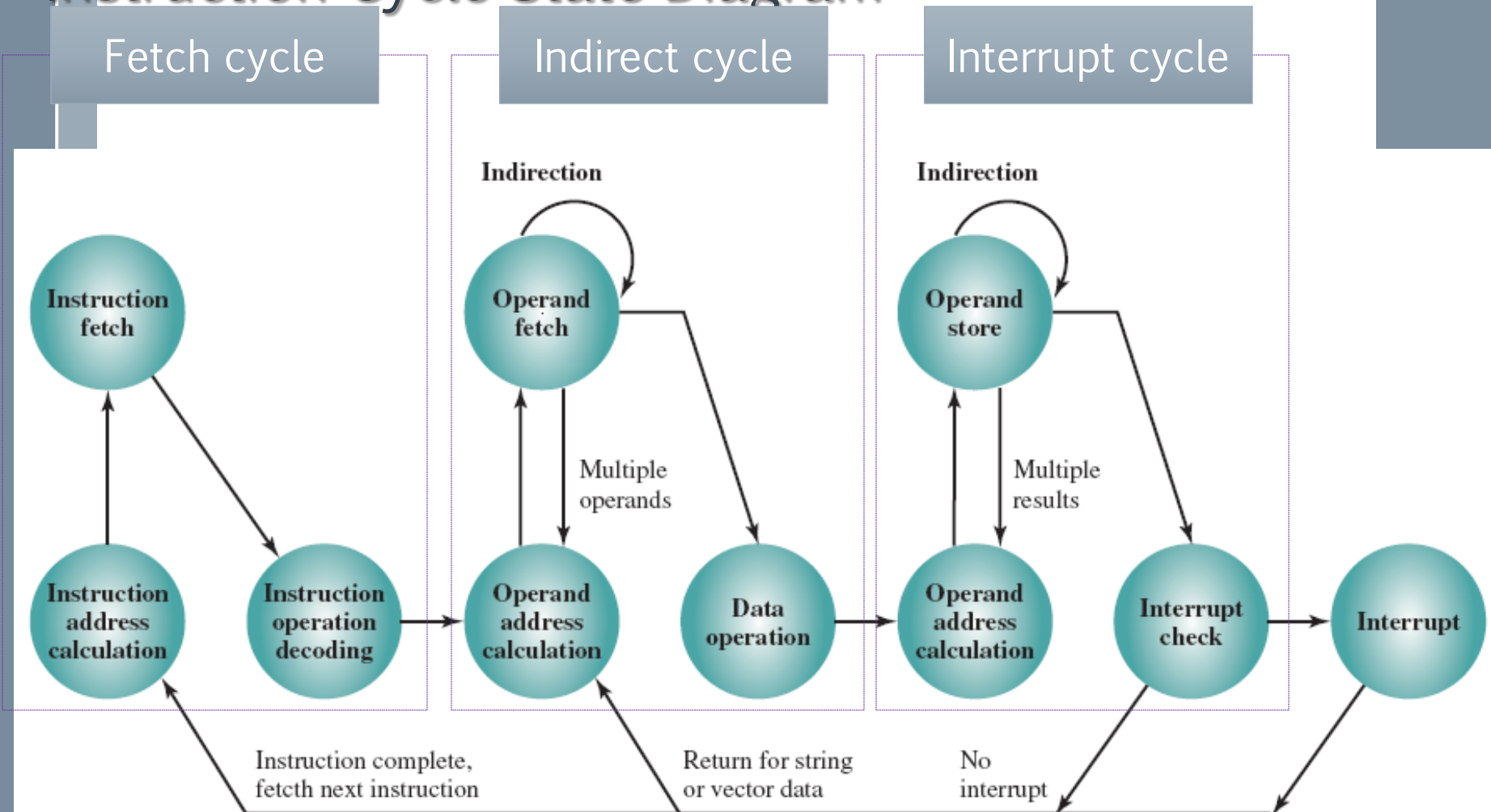
If interrupts are enabled and an interrupt has occurred, **save the current process state** and **service the interrupt**

# Instruction Cycle



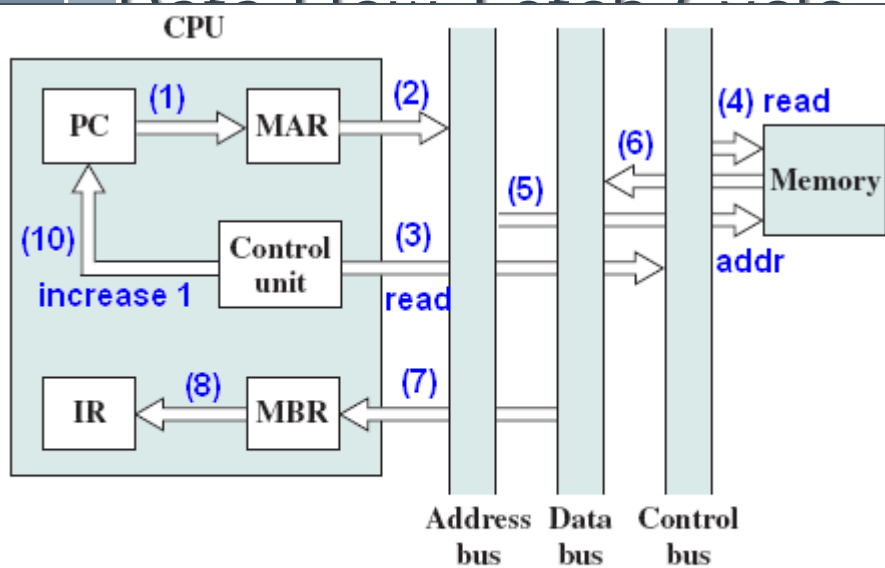
**Figure 14.4 The Instruction Cycle**

# Instruction Cycle State Diagram



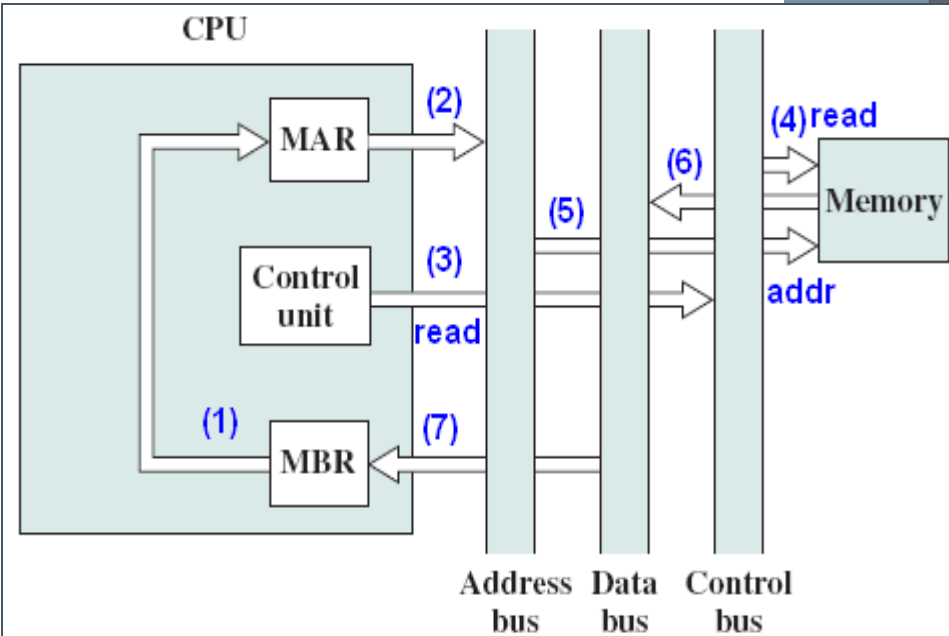
**Figure 14.5** Instruction Cycle State Diagram

## Data Flow, Fetch Cycle



**Figure 14.6** Data Flow, Fetch Cycle

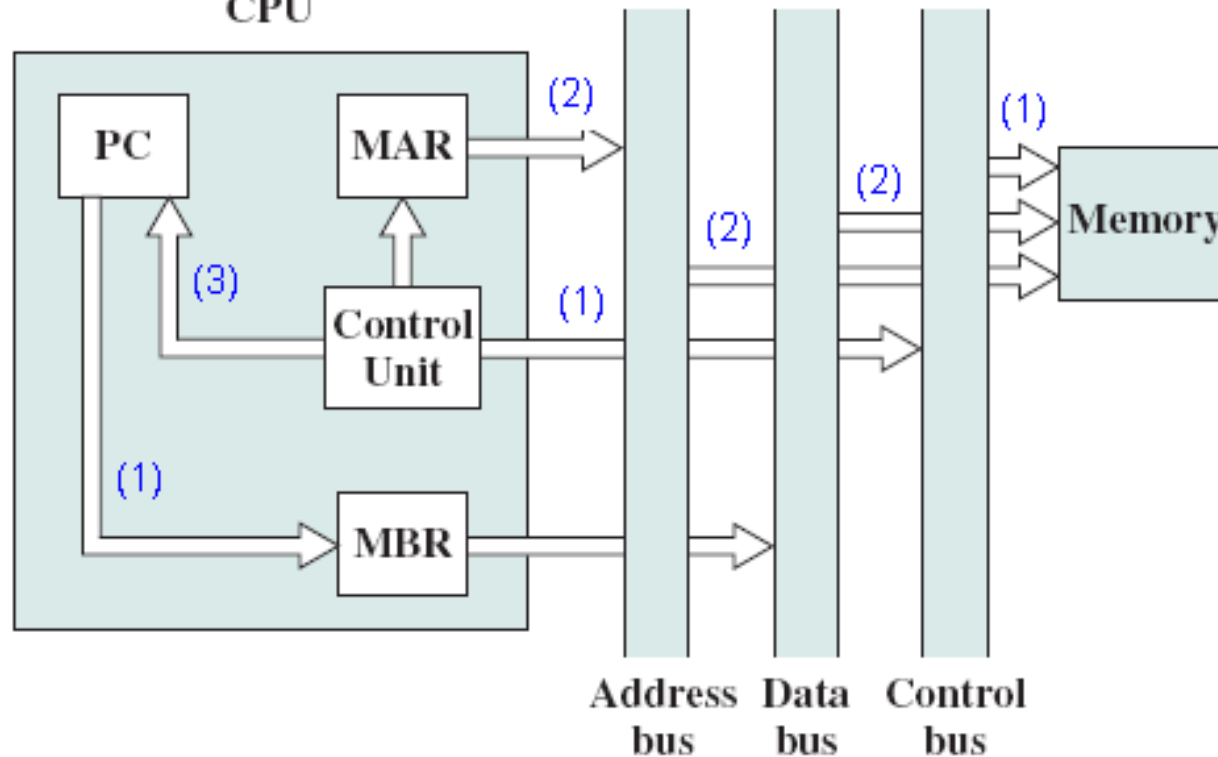
Fetch cycle for the next instruction  
(Instruction index is in PC)  
MAR: Memory Address Register  
MBR: Memory buffer Register



**Figure 14.7** Data Flow, Indirect Cycle

The CU examines the contents of the IR to determine if it contains an operand specified by indirect addressing → Use indirect cycle (data address is in MBR)

## Data Flow, Interrupt Cycle



**Figure 14.8** Data Flow, Interrupt Cycle

- (1) Store PC (return point after executing interrupt routine)
- (2) Store current state (values in registers before running interrupt routine)
- (3) Fetch cycle is used to load interrupt routine



## 14.4- Instruction Pipelining Pipelining Strategy

A way to improve performance is performing jobs in parallel manner

Similar to the use of an assembly line in a manufacturing plant

To apply this concept to instruction execution we must recognize that an **instruction has a number of stages**



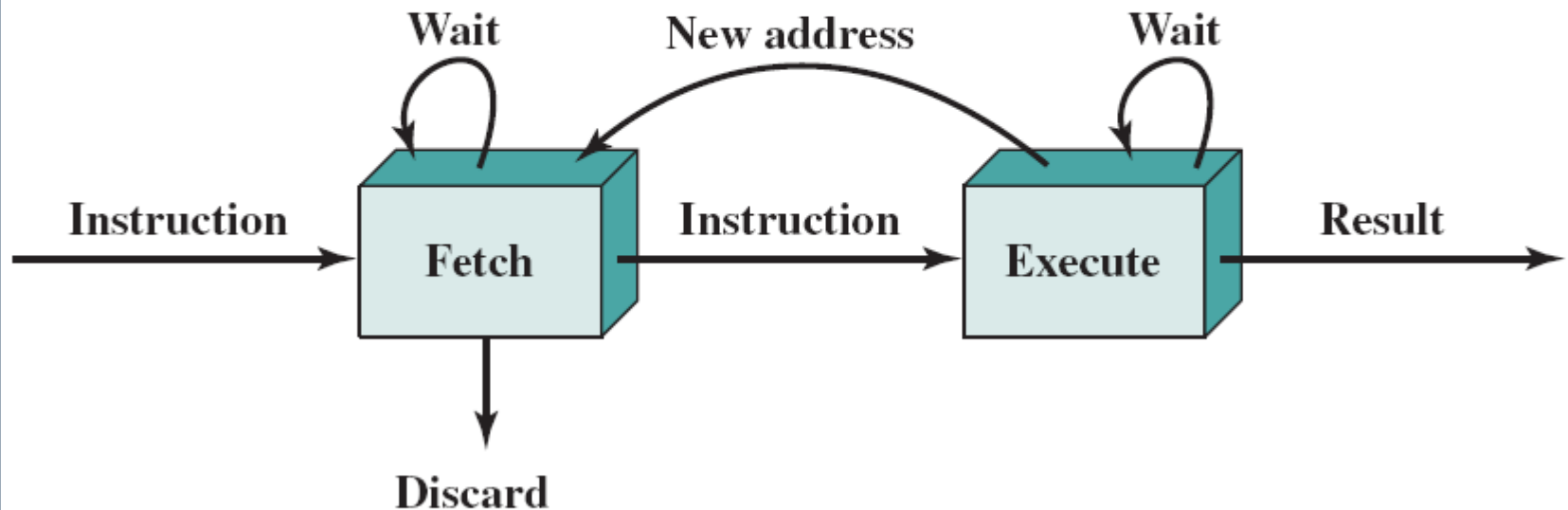
**New inputs are accepted at one end before previously accepted inputs appear as outputs at the other end**

An assembly line (dây chuyền xử lý) in which some operations are performed concurrently

## Two-Stage Instruction Pipeline



(a) Simplified view



(b) Expanded view

**Figure 14.9** Two-Stage Instruction Pipeline

# Additional Stages

$\pi$

- › **Fetch instruction (FI)**
  - Read the next expected instruction into a buffer
- › **Decode instruction (DI)**
  - Determine the opcode and the operand specifiers
- › **Calculate operands (CO)**
  - Calculate the effective address of each source operand
  - This may involve displacement, register indirect, indirect, or other forms of address calculation
- › **Fetch operands (FO)**
  - Fetch each operand from memory
  - Operands in registers need not be fetched
- › **Execute instruction (EI)**
  - Perform the indicated operation and store the result, if any, in the specified destination operand location
- › **Write operand (WO)**
  - Store the result in memory

# Timing Diagram for Instruction Pipeline Operation

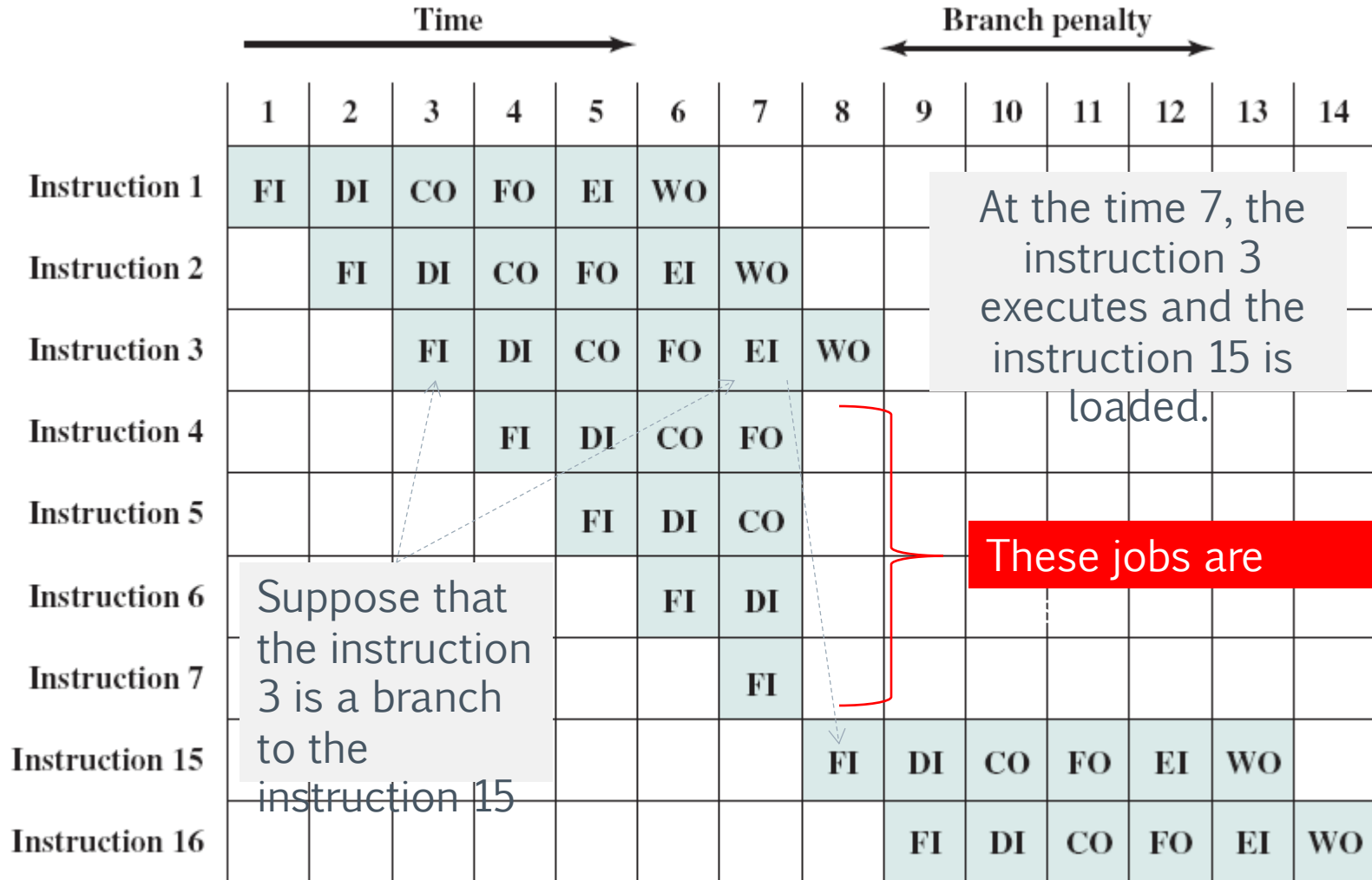
Time →

|               | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
|---------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Instruction 1 | FI | DI | CO | FO | EI | WO |    |    |    |    |    |    |    |    |
| Instruction 2 |    | FI | DI | CO | FO | EI | WO |    |    |    |    |    |    |    |
| Instruction 3 |    |    | FI | DI | CO | FO | EI | WO |    |    |    |    |    |    |
| Instruction 4 |    |    |    | FI | DI | CO | FO | EI | WO |    |    |    |    |    |
| Instruction 5 |    |    |    |    | FI | DI | CO | FO | EI | WO |    |    |    |    |
| Instruction 6 |    |    |    |    |    | FI | DI | CO | FO | EI | WO |    |    |    |
| Instruction 7 |    |    |    |    |    |    | FI | DI | CO | FO | EI | WO |    |    |
| Instruction 8 |    |    |    |    |    |    |    | FI | DI | CO | FO | EI | WO |    |
| Instruction 9 |    |    |    |    |    |    |    |    | FI | DI | CO | FO | EI | WO |

I: Instruction  
 O: operand  
 F: Fetch  
 C: Calculate  
 F: Fetch  
 E: Execute  
 W: Write

**Figure 14.10** Timing Diagram for Instruction Pipeline Operation

# The Effect of a Conditional Branch on Instruction Pipeline Operation



**Figure 14.11** The Effect of a Conditional Branch on Instruction Pipeline Operation

# SIX STAGE INSTRUCTION PIPELINE

Figure 14.12 indicates the logic needed for pipelining to account for branches and interrupts

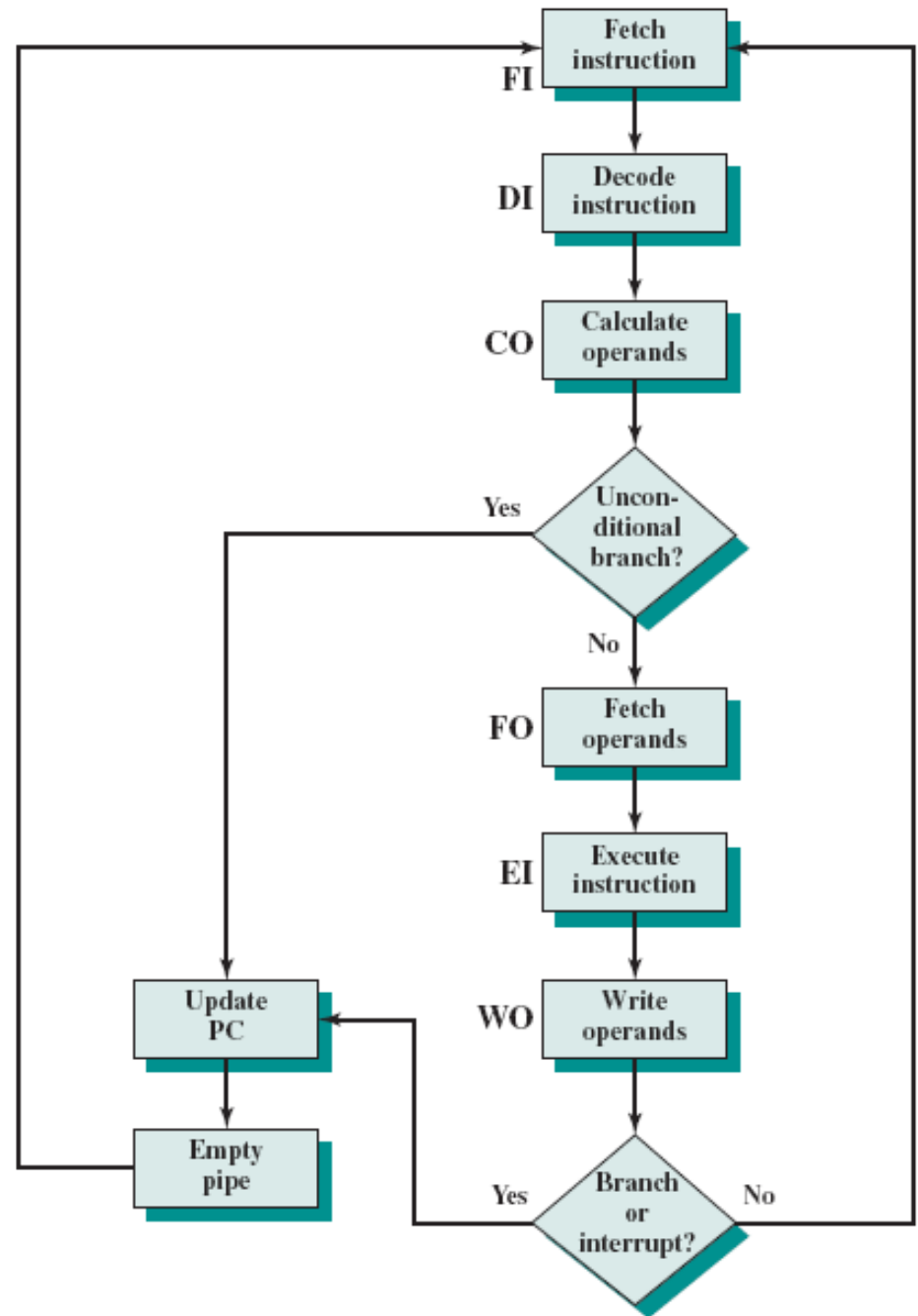


Figure 14.12 Six-Stage CPU Instruction Pipeline

Time ↓

|    | FI | DI | CO | FO | EI | WO |
|----|----|----|----|----|----|----|
| 1  | I1 |    |    |    |    |    |
| 2  | I2 | I1 |    |    |    |    |
| 3  | I3 | I2 | I1 |    |    |    |
| 4  | I4 | I3 | I2 | I1 |    |    |
| 5  | I5 | I4 | I3 | I2 | I1 |    |
| 6  | I6 | I5 | I4 | I3 | I2 | I1 |
| 7  | I7 | I6 | I5 | I4 | I3 | I2 |
| 8  | I8 | I7 | I6 | I5 | I4 | I3 |
| 9  | I9 | I8 | I7 | I6 | I5 | I4 |
| 10 |    | I9 | I8 | I7 | I6 | I5 |
| 11 |    |    | I9 | I8 | I7 | I6 |
| 12 |    |    |    | I9 | I8 | I7 |
| 13 |    |    |    |    | I9 | I8 |
| 14 |    |    |    |    |    | I9 |

(a) No branches

|    | FI  | DI  | CO  | FO  | EI  | WO  |
|----|-----|-----|-----|-----|-----|-----|
| 1  | I1  |     |     |     |     |     |
| 2  | I2  | I1  |     |     |     |     |
| 3  | I3  | I2  | I1  |     |     |     |
| 4  | I4  | I3  | I2  | I1  |     |     |
| 5  | I5  | I4  | I3  | I2  | I1  |     |
| 6  | I6  | I5  | I4  | I3  | I2  | I1  |
| 7  | I7  | I6  | I5  | I4  | I3  | I2  |
| 8  | I15 |     |     |     |     | I3  |
| 9  | I16 | I15 |     |     |     |     |
| 10 |     | I16 | I15 |     |     |     |
| 11 |     |     | I16 | I15 |     |     |
| 12 |     |     |     | I16 | I15 |     |
| 13 |     |     |     |     | I16 | I15 |
| 14 |     |     |     |     |     | I16 |

(b) With conditional branch

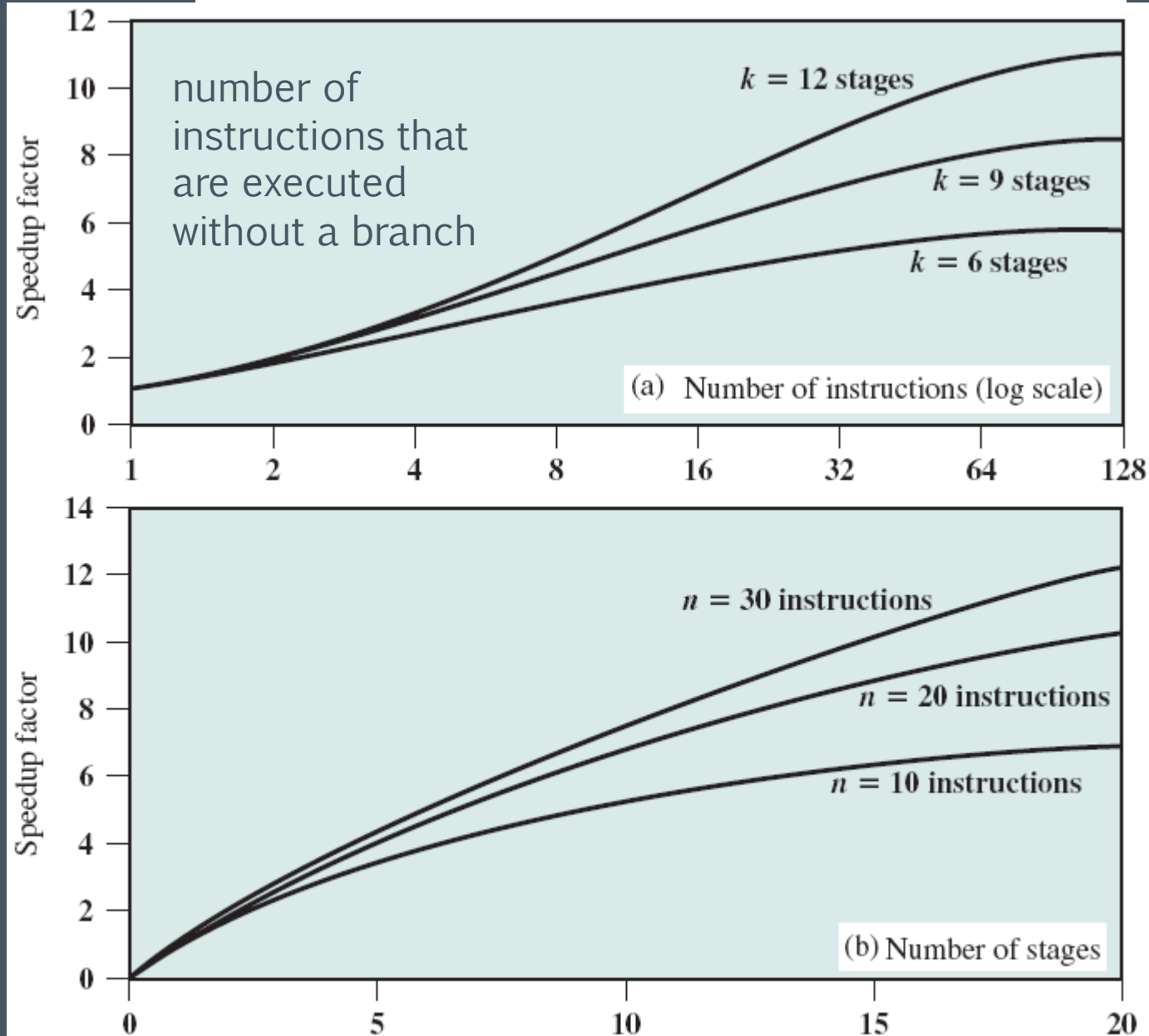
ALTERNATIVE  
PIPELINE  
DEPICTION

I3 is a  
conditional  
branch to I15

**Figure 14.13** An Alternative Pipeline Depiction

# SPEEDUP FACTORS WITH INSTRUCTION PIPELINING

The larger the number of pipeline stages, the greater the potential for speedup → **HIGHER COST**



**Figure 14.14** Speedup Factors with Instruction Pipelining

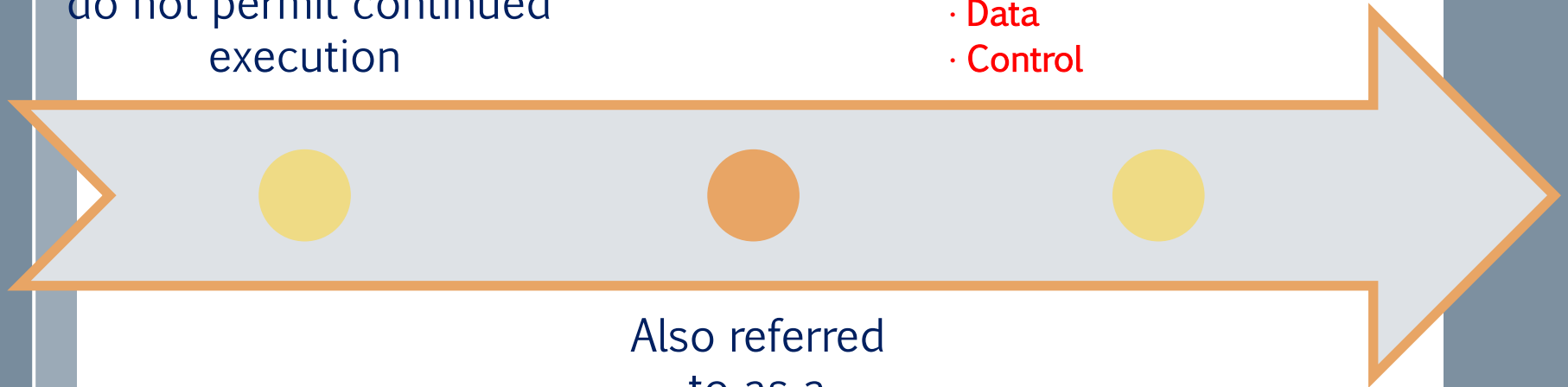


# Pipeline Hazards (rủi ro)

Occur when the pipeline, or **some portion** of the pipeline, must **stall** (trì hoãn) because conditions do not permit continued execution

There are three **types** of hazards:

- **Resource**
- **Data**
- **Control**



Also referred to as a ***pipeline bubble***



# RESOURCE HAZARDS

A resource hazard occurs when **two or more instructions** that are already in the pipeline **need the same resource**

The **result** is that the instructions must be **executed in serial** rather than parallel for a portion of the pipeline

A **resource hazard** is sometimes referred to as a **structural hazard**

|             |    | Clock cycle |    |    |    |    |    |    |    |   |
|-------------|----|-------------|----|----|----|----|----|----|----|---|
|             |    | 1           | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9 |
| Instruction | I1 | FI          | DI | FO | EI | WO |    |    |    |   |
|             | I2 |             | FI | DI | FO | EI | WO |    |    |   |
|             | I3 |             |    | FI | DI | FO | EI | WO |    |   |
|             | I4 |             |    |    | FI | DI | FO | EI | WO |   |

(a) Five-stage pipeline, ideal case

|             |    | Clock cycle |    |      |    |    |    |    |    |    |
|-------------|----|-------------|----|------|----|----|----|----|----|----|
|             |    | 1           | 2  | 3    | 4  | 5  | 6  | 7  | 8  | 9  |
| Instruction | I1 | FI          | DI | FO   | EI | WO |    |    |    |    |
|             | I2 |             | FI | DI   | FO | EI | WO |    |    |    |
|             | I3 |             |    | Idle | FI | DI | FO | EI | WO |    |
|             | I4 |             |    |      |    | FI | DI | FO | EI | WO |

(b) I1 source operand in memory

Figure 14.15 Example of Resource Hazard

FO is accessing memory. So, this step is idle

# DATA HAZARDS

A data hazard occurs when there is a conflict in the access of an operand location

RAW

Instruction is executing and the register EAX is writing to. So, it can not be read.

Hazard

X86  
instruction

|     |          | Clock cycle |    |    |      |    |    |    |    |    |    |
|-----|----------|-------------|----|----|------|----|----|----|----|----|----|
|     |          | 1           | 2  | 3  | 4    | 5  | 6  | 7  | 8  | 9  | 10 |
| ADD | EAX, EBX | FI          | DI | FO | EI   | WO |    |    |    |    |    |
| SUB | ECX, EAX |             | FI | DI | Idle | FO | EI | WO |    |    |    |
| I3  |          |             |    | FI |      | DI | FO | EI | WO |    |    |
| I4  |          |             |    |    |      | FI | DI | FO | EI | WO |    |

Figure 14.16 Example of Data Hazard

# $\pi$ Types of Data Hazard

- › **Read after write (RAW), or true dependency**
  - An instruction modifies a register or memory location
  - Succeeding instruction reads data in memory or register location
  - Hazard occurs if the **read takes place before write** operation is complete
- › **Write after read (WAR), or antidependency**
  - An instruction reads a register or memory location
  - Succeeding instruction writes to the location
  - Hazard occurs if the **write** operation completes **before** the **read** operation takes place
- › **Write after write (WAW), or output dependency**
  - Two instructions both write to the same location
  - Hazard occurs if the write operations take place in the **reverse order** of the intended sequence

# Control Hazard

- › Also known as a *branch hazard*
- › Occurs when the pipeline makes the wrong decision on a branch prediction
- › Brings instructions into the pipeline that must subsequently be discarded
- › Dealing with Branches:
  - Multiple streams
  - Prefetch branch target
  - Loop buffer
  - Branch prediction
  - Delayed branch

# Multiple Streams

A simple pipeline suffers a penalty for a branch instruction because it must choose one of two instructions to fetch next and may make the wrong choice

A brute-force approach is to replicate the initial portions of the pipeline and allow the pipeline to fetch both instructions, making use of two streams

brute-force  
search or exhaustive search  
(vét cạn)

## Drawbacks:

- With multiple pipelines there are contention delays for access to the registers and to memory
- Additional branch instructions may enter the pipeline before the original branch decision is resolved

# PREFETCH BRANCH TARGET

- When a conditional branch is recognized, the target of the branch is prefetched, in addition to the instruction following the branch
- Target is then saved until the branch instruction is executed
- If the branch is taken, the target has already been prefetched
- IBM 360/91 uses this approach

# Loop Buffer

Small, very-high speed memory maintained by the instruction fetch stage of the pipeline and containing the  $n$  most recently fetched instructions, in sequence

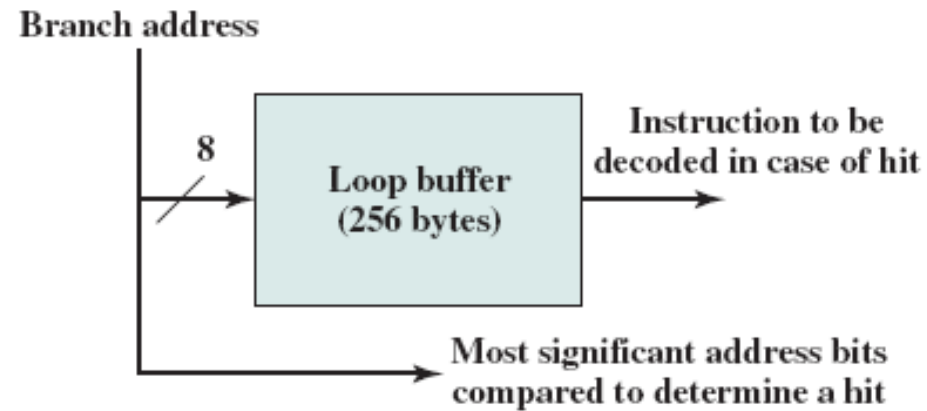


Figure 14.17 Loop Buffer

## › Benefits:

- Instructions fetched in sequence will be available without the usual memory access time
- If a branch occurs to a target just a few locations ahead of the address of the branch instruction, the target will already be in the buffer
- This strategy is particularly well suited to dealing with loops

Similar in principle to a cache dedicated to instructions. Differences:

- The loop buffer only retains instructions in sequence
- Is much smaller in size and hence lower in cost



## Branch Prediction

› Various techniques can be used to predict whether a branch will be taken:

1. Predict never taken
2. Predict always taken
3. Predict by opcode

■ **These approaches are static**

■ They do not depend on the execution history up to the time of the conditional branch instruction

1. Taken/not taken switch
2. Branch history table

■ **These approaches are dynamic**

■ They depend on the execution history

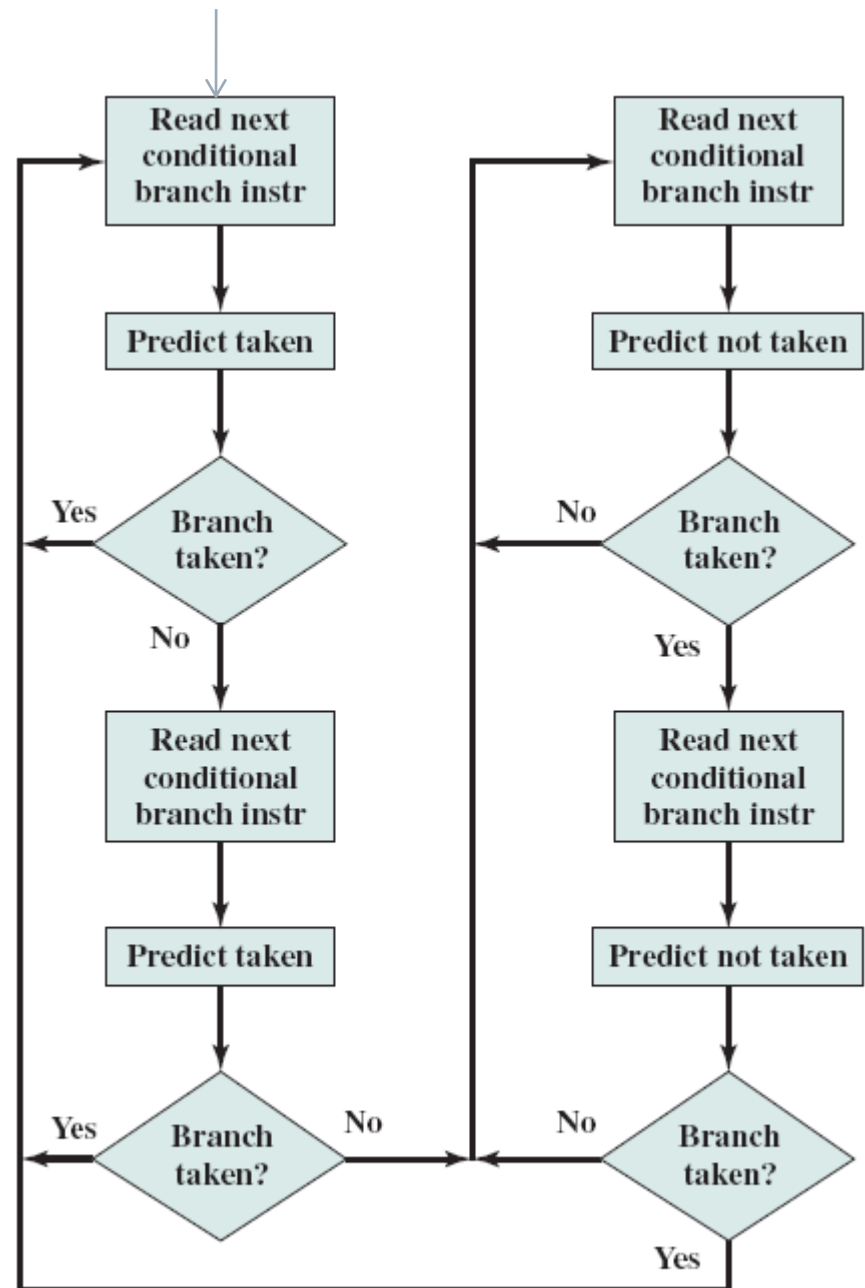
How are predictions carried out?  
Next slide

➔ States of some last instructions  
(some bits) must be stores in  
cache

# BRANCH PREDICTION FLOW CHART

If only one bit is stored, a loop may cause 2 errors in prediction: once on entering and once on exiting.

If 2 bits are stored, a prediction algorithm is carried out using 2 branches (fig. 14.18)

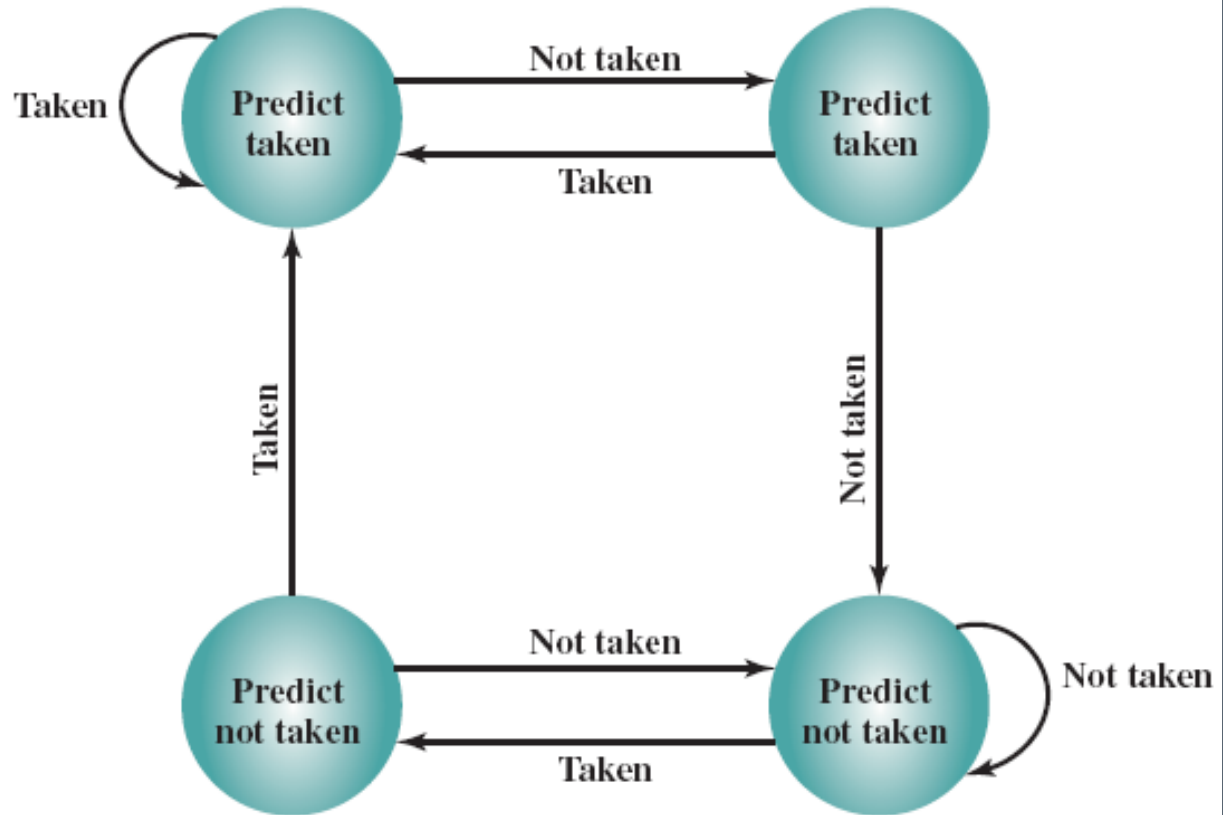


**Figure 14.18** Branch Prediction Flowchart

# Branch Prediction State Diagram

The decision process can be represented more compactly by a finite-state machine

Finite-state machine is a way to express a processing mechanism in which each part of input will determine a step of the process.



**Figure 14.19** Branch Prediction State Diagram

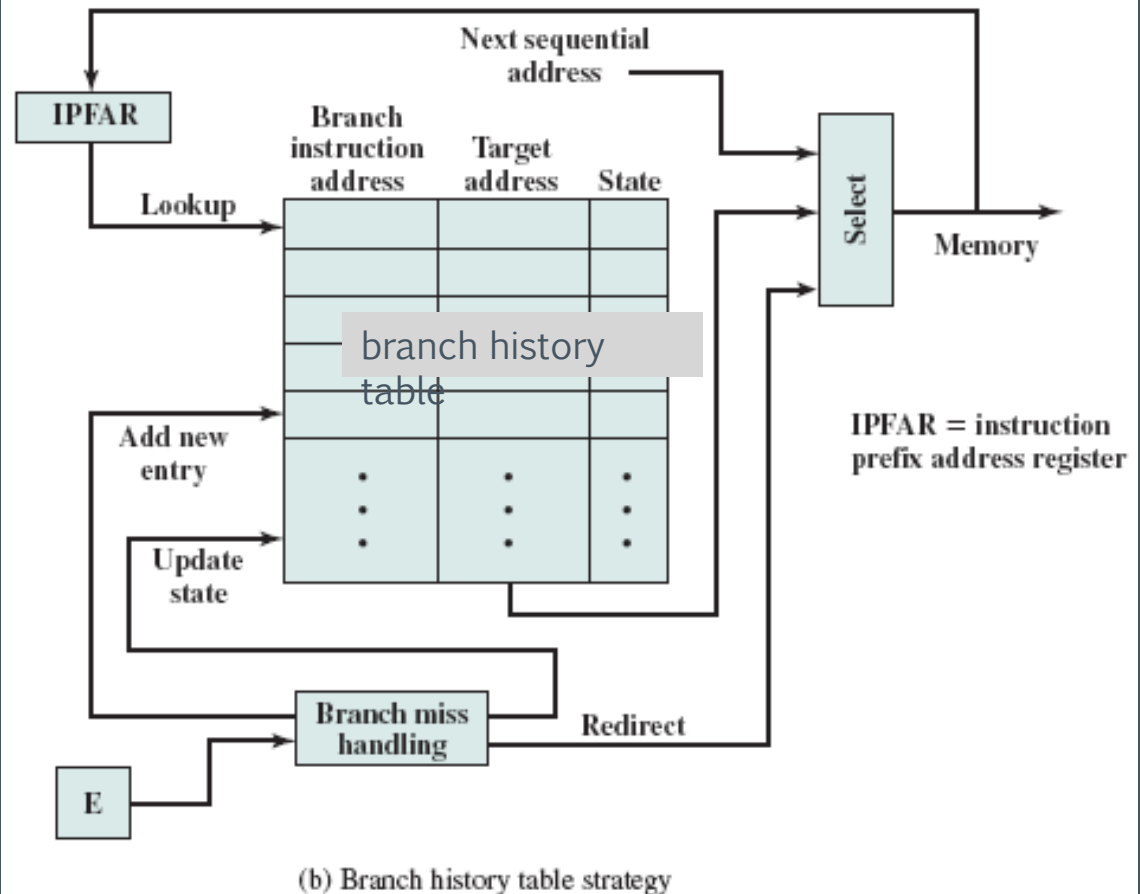
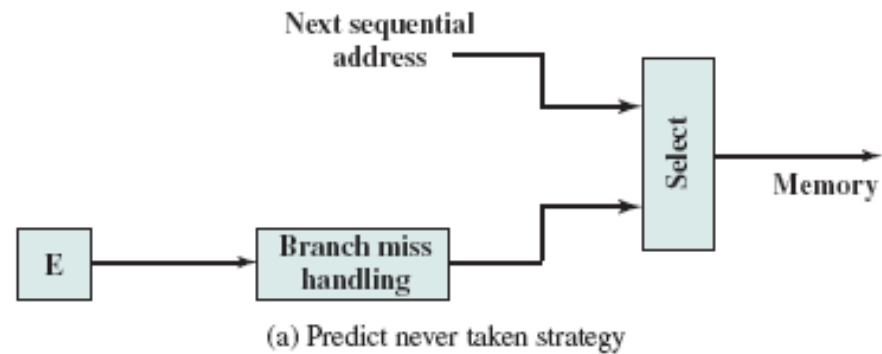
Some bits are stored: 0: Not taken, 1: Taken. A history can be as 01110

# DEALING WITH BRANCHES

Each prefetch triggers a lookup in the table.

**No match:** Fetch next sequential address.

**Match:** a prediction is made based on the state of the instruction: Either the next sequential address or the branch target address is fed to the select logic.



**Figure 14.20** Dealing with Branches

## Delayed Branch

$\pi$

- › It is possible to improve pipeline performance by **automatically rearranging instructions** within a program, so that branch instructions occur later than actually desired. This intriguing approach is examined in Chapter 15.

# Intel 80486 Pipelining

$\pi$

## › Fetch

- Objective is to **fill the prefetch buffers** with new data as soon as the old data have been consumed by the instruction decoder
- **Operates independently** of the other stages to keep the prefetch buffers full

## › Decode stage 1

- All **opcode** and addressing-mode information is decoded in the D1 stage
- **3 bytes** of instruction are passed to the D1 stage from **the prefetch buffers**
- D1 decoder can then **direct the D2 stage** to capture the rest of the instruction

## › Decode stage 2

- Expands each opcode into control signals for the ALU
- Also controls the computation of the more complex addressing modes

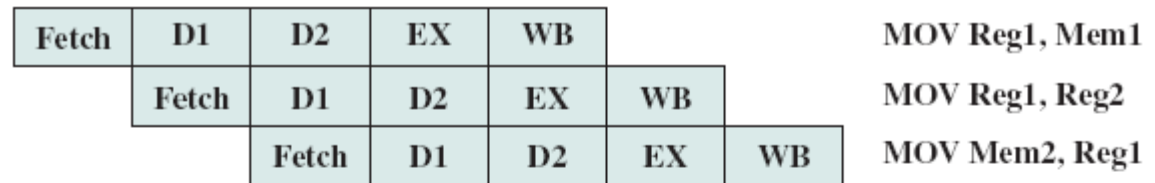
## › Execute

- Stage includes ALU operations, cache access, and register update

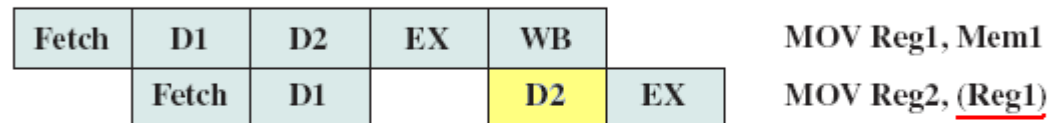
## › Write back

- **Updates registers** and **status flags** modified during the preceding execute stage

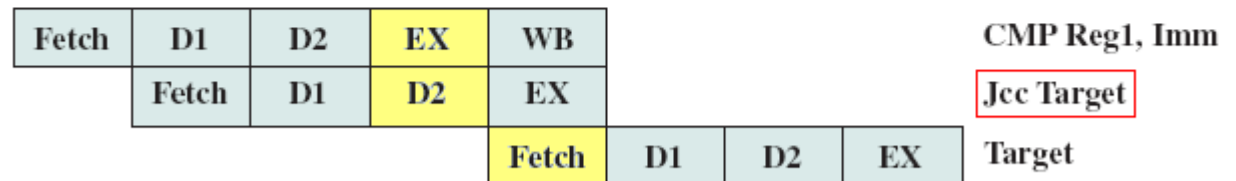
# 80486 INSTRUCTION PIPELINE EXAMPLES



(a) No data load delay in the pipeline



(b) Pointer load delay



(c) Branch instruction timing

### Figure 14.21 80486 Instruction Pipeline Examples



## Exercises

- › 14.1 What general roles are performed by processor registers?
- › 14.2 What categories of data are commonly supported by user-visible registers?
- › 14.3 What is the function of condition codes?
- › 14.4 What is a program status word?
- › 14.5 Why is a two-stage instruction pipeline unlikely to cut the instruction cycle time in half, compared with the use of no pipeline?
- › 14.6 List and briefly explain various ways in which an instruction pipeline can deal with conditional branch instructions.
- › 14.7 How are history bits used for branch prediction?





## Exercises

14.8 - what would be the value of the following flags: Carry, Zero, Overflow, Sign, Even Parity , Half-Carry ?

- › (a) If the last operation performed on a computer with an 8-bit word was an addition in which the two operands were 00000010 and 00000011.
- › (b) Repeat for the addition of -1 (twos complement) and +1.
- › (c) A - B, where A contains 11110000 and B contains 0010100.

# $\pi$ Summary

## CHAPTER 7

- › Processor organization
- › Register organization
  - User-visible registers
  - Control and status registers
- › Instruction cycle
  - The indirect cycle
  - Data flow

## PROCESSOR STRUCTURE AND FUNCTION

- › Instruction pipelining
  - Pipelining strategy
  - Pipeline performance
  - Pipeline hazards
  - Dealing with branches
  - Intel 80486 pipelining